IsalSR: Instruction Set and Language for Symbolic Regression

Technical Report — Implementation and Experimental Design

Ezequiel Lopez-Rubio[1] Mario Pascual Gonzalez[1]

[1]University of Malaga, Spain
{ezeqlr, mpascual}@uma.es

March 2026

Abstract

We present IsalSR (Instruction Set and Language for Symbolic Regression), a representation framework that encodes mathematical expressions as labeled Directed Acyclic Graphs (DAGs) via isomorphism-invariant instruction strings. By computing a canonical string—a complete labeled-DAG invariant—IsalSR collapses $O(k!)$ equivalent expression representations into a single canonical form, where k is the number of internal (non-variable) nodes. This reduction enables existing symbolic regression methods to search over a dramatically smaller space without modifying their core algorithms. This report describes the instruction set design, canonical string computation, software architecture, and experimental infrastructure for validating the $O(k!)$ claim on standard SR benchmarks (Nguyen, Feynman, SRBench). The implementation comprises 286 verified tests across 7 development phases, with 78/80 proposal-alignment checks passed.

1 Proposal and Positioning

1.1 The Problem: Redundancy in Symbolic Regression Search Spaces

Symbolic regression (SR) seeks a mathematical expression $f(x_1, \ldots, x_m)$ that best fits observed data (X, y). Modern approaches represent expressions as Directed Acyclic Graphs (DAGs) where leaf nodes are variables or constants, and internal nodes are operations $(+, \times, \sin, \cos, \text{etc.})$ [2, 4].

A fundamental issue plagues all existing SR methods: the search space contains **massive redundancy**. For an expression DAG with k internal nodes, there exist $O(k!)$ distinct node labelings that all represent the *same expression*. Each of these $k!$ variants occupies a separate point in the search space, wasting computational effort on equivalent candidates.

Proposition 1 (Search space redundancy)**.** *For an expression DAG D with k internal (non-variable) nodes, the number of structurally equivalent but differently-labeled DAGs is $\Theta(k!)$ in the worst case.*

1.2 Our Contribution: Isomorphism-Invariant Representation

IsalSR extends the IsalGraph framework [1]—originally designed for topology-only unlabeled graphs—to **labeled DAGs** for symbolic regression. The key contribution is the **canonical string** w_D^*: a complete labeled-DAG invariant that satisfies

$$w_{D_1}^* = w_{D_2}^* \iff D_1 \cong D_2 \tag{1}$$

where $\cong$ denotes labeled-DAG isomorphism (preserving node labels and edge directions).

By mapping every expression to its canonical string before evaluation, the search space is reduced from $O(N \cdot k!)$ to $O(N)$ unique expressions, where N is the number of structurally

distinct DAGs. This is the **sole contribution**: we provide an invariant representation that existing SR methods can plug into for acceleration.

1.3 What IsalSR Is NOT

Following the advisor's explicit guidance, we emphasize:

- IsalSR is **not a new SR method**. It is a representation layer.
- We do **not claim SR expertise**. Our expertise is in graph theory and combinatorics applied to expression representation.
- The search operators (mutation, crossover, hill climbing, evolutionary population) are **standard, generic wrappers**—not novel algorithms.
- The experimental design compares the **same algorithm** operating in canonical vs. non-canonical space, not different algorithms.

1.4 Relationship to IsalGraph

IsalGraph [1] represents arbitrary *unlabeled, undirected* graphs as instruction strings. IsalSR adapts this framework with four key modifications:

1. **Labeled nodes**: Each node has a type (VAR, ADD, MUL, SIN, etc.) encoded in two-character compound tokens.
2. **Directed edges**: Edges represent data flow ($u \to v$ means "u provides input to v").
3. **DAG constraint**: The C/c edge instructions enforce acyclicity via cycle detection (silently skipping cycle-creating edges).
4. **Fixed starting node**: The canonical string is computed from x_1 only (the first input variable), since variables are distinguishable and pre-numbered. This eliminates the $O(n)$ loop over starting nodes present in IsalGraph.

2 Instruction Set and Core Structures

2.1 The IsalSR Alphabet Σ_{SR}

IsalSR uses a **two-tier token encoding**: single-character tokens for movement and edge operations, and two-character compound tokens for labeled node insertion.

Definition 1 (IsalSR Alphabet)**.** *The alphabet Σ_{SR} consists of:*

- ***Movement*** *(single-char): $\{N, P, n, p\}$ — move primary/secondary pointer next/prev in the CDLL.*
- ***Edge creation*** *(single-char): $\{C, c\}$ — directed edge primary→secondary / secondary→primary, with DAG cycle check.*
- ***No-op*** *(single-char): $\{W\}$.*
- ***Node insertion*** *(two-char): $V\ell$ or $v\ell$ where ℓ is a label character from $\mathcal{L} = \{+, *, -, /, s, c, e, l, r, \hat{}, a, k\}$.*

Total vocabulary: 7 single-char + 24 compound = 31 tokens (configurable subset per experiment).

2.2 Initial State

For m input variables $x_1, \ldots, x_m$:

- The DAG contains m VAR nodes (IDs $0, 1, \ldots, m-1$), no edges.
- The CDLL contains m entries $[x_1, x_2, \ldots, x_m]$ in order.
- Both pointers (primary, secondary) start on the CDLL node for x_1.

Variables are **pre-numbered and distinguishable**: $x_1 \neq x_2$ under isomorphism. This eliminates isomorphism ambiguity over variable assignments and is the key simplification that allows the canonical string to be computed from x_1 only.

Table 1: Operation types and their label characters.

Label	NodeType	Arity	Description
$+$	ADD	Variable (≥ 2)	Sum of all inputs
$*$	MUL	Variable (≥ 2)	Product of all inputs
$-$	SUB	Binary (2)	Subtraction
$/$	DIV	Binary (2)	Protected division
s	SIN	Unary (1)	Sine
c	COS	Unary (1)	Cosine
e	EXP	Unary (1)	Protected exponential
l	LOG	Unary (1)	Protected logarithm
r	SQRT	Unary (1)	Protected square root
ˆ	POW	Binary (2)	Protected power
a	ABS	Unary (1)	Absolute value
k	CONST	Leaf (0)	Learnable constant

2.3 Edge Semantics and the DAG Constraint

An edge $u \to v$ means "u provides input to v" (data flow direction). For $\sin(x)$: edge $x \to \sin$. For $x + y$: edges $x \to +$ and $y \to +$.

The C instruction creates a directed edge from the primary pointer's graph node to the secondary pointer's graph node. Before adding any edge via C/c, the system checks whether the edge would create a cycle (via BFS reachability). If a cycle would result, the instruction is **silently skipped**. V/v insertions never create cycles because new nodes have no outgoing edges.

2.4 Core Data Structures

LabeledDAG: Directed acyclic graph with dual adjacency lists (in-edges and out-edges), node labels (`NodeType` enum), and per-node metadata (variable index, constant value). Supports $O(V + E)$ cycle detection, Kahn's topological sort, and label-aware backtracking isomorphism.

CircularDoublyLinkedList (CDLL): Array-backed circular list with $O(1)$ insert-after, remove, and traversal. Reused verbatim from IsalGraph [1]. Pointers are CDLL node indices (not graph node IDs); conversion via `get_value(ptr)`.

3 Core Algorithms

3.1 StringToDAG (S2D) — The Decoder

The S2D decoder executes an IsalSR instruction string token-by-token to build a `LabeledDAG`. It tokenizes the input using a greedy scanner (consuming two characters for V/v compound tokens) and dispatches each token to update the shared state: (DAG, CDLL, primary pointer, secondary pointer).

3.2 DAGToString (D2S) — The Greedy Encoder

The D2S encoder converts a `LabeledDAG` back into an instruction string using a greedy algorithm adapted from IsalGraph [1]. It searches for the minimum-cost pointer displacement that enables a structural operation (V, v, C, or c) at each step.

The displacement pairs (a, b) are sorted by $|a| + |b|$ (total movement cost), ensuring the shortest possible movement instructions are emitted first. The priority order is $V > v > C > c$: node insertion is preferred over edge-only insertion.

Key adaptations from IsalGraph:

Algorithm 1 StringToDAG (S2D)

Require: String w, number of variables m

```
Initialize DAG with m VAR nodes; CDLL ← [x_1, ..., x_m]
pri ← CDLL node for x_1; sec ← pri
for each token t in Tokenize(w) do
    if t ∈ {N, P, n, p} then move corresponding pointer
    else if t = Vℓ then create node with label ℓ, edge pri→new, insert after pri
    else if t = vℓ then same from secondary pointer
    else if t = C then add edge pri→sec if acyclic
    else if t = c then add edge sec→pri if acyclic
    end if
end for
return DAG
```

- Token emission: `"V" + label_char` (two-character compound tokens).
- Neighbor search: `out_neighbors()` only (directed data flow).
- All m VAR nodes are pre-mapped at initialization.

3.3 Round-Trip Property

Theorem 1 (Round-trip)**.** *For any valid IsalSR string w and number of variables m:*

$$S2D(w, m) \cong S2D(D2S(S2D(w, m), x_1), m)$$

where $\cong$ denotes labeled-DAG isomorphism.

This property is verified by 25 deterministic tests and Hypothesis property tests across random strings.

3.4 Canonical String — The Core Contribution

Definition 2 (Canonical string)**.** *For a labeled DAG D with initial node x_1:*

$$w_D^* = lexmin\left\{w \in \arg\min_{P \in Paths(D,x_1)} |P|\right\}$$

The shortest D2S string from x_1, with ties broken lexicographically. Computed via exhaustive backtracking over all valid neighbor choices at V/v branch points.

Theorem 2 (Complete invariant)**.** *$w_{D_1}^* = w_{D_2}^*$ if and only if $D_1 \cong D_2$ (labeled-DAG isomorphism).*

Simplification from IsalGraph: Since input variables are distinguishable (x_i maps to x_i under any isomorphism), the canonical string is computed from x_1 only—no iteration over starting nodes. This reduces the canonical computation cost by a factor of $O(n)$.

3.5 Pruned Canonical with 6-Component Structural Tuples

At each V/v branch point, only candidates sharing the maximum 6-component structural tuple are explored:

$$\tau(v) = (|\text{in}_{N_1}(v)|, |\text{out}_{N_1}(v)|, |\text{in}_{N_2}(v)|, |\text{out}_{N_2}(v)|, |\text{in}_{N_3}(v)|, |\text{out}_{N_3}(v)|)$$

where $\text{in}_{N_k}(v)$ and $\text{out}_{N_k}(v)$ count nodes at distance k following incoming and outgoing edges, respectively. This is computed via truncated BFS in both directions.

The pruned variant preserves the complete invariant property while dramatically reducing the branching factor (empirically 10–100× faster).

3.6 DAG Distance Metric

The Levenshtein distance between canonical strings defines a metric on labeled DAGs:

$$d(D_1, D_2) = \text{Levenshtein}(w^*_{D_1}, w^*_{D_2})$$

This satisfies symmetry, triangle inequality, and identity of indiscernibles.

4 Software Architecture

4.1 Dependency Layering

IsalSR enforces strict dependency isolation:

```
experiments/, benchmarks/  -> anything (numpy, scipy, matplotlib, ...)
isalsr.search              -> numpy
isalsr.evaluation          -> numpy, scipy
isalsr.adapters            -> optional: networkx, sympy
isalsr.core                -> ZERO external dependencies (stdlib only)
```

The core layer (`isalsr.core`) has zero external dependencies, enabling lightweight distribution and reproducibility.

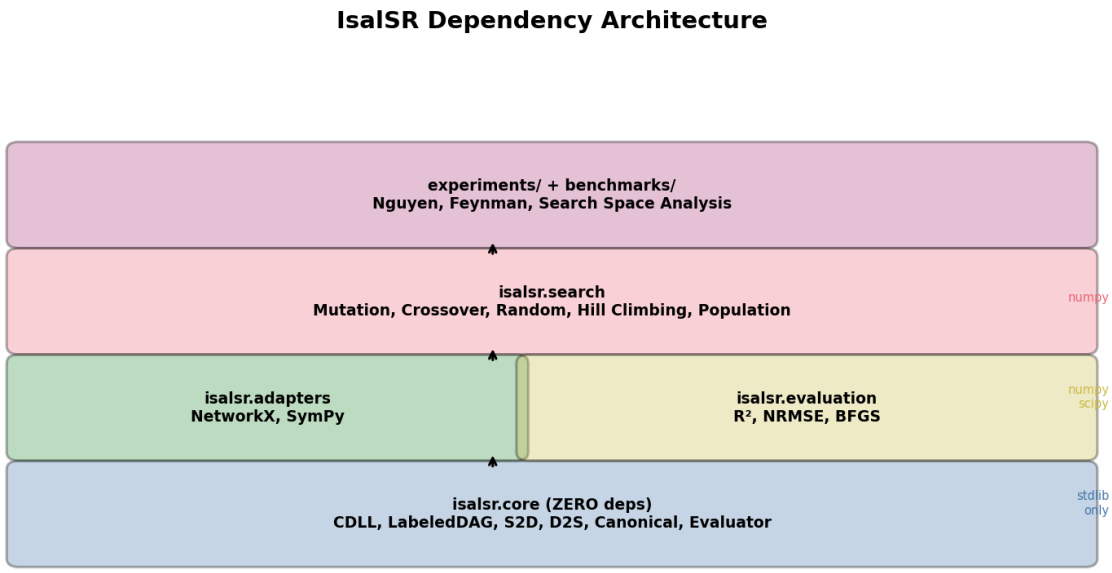

Figure 1: IsalSR dependency architecture. Each layer only depends on layers below it. The core layer has zero external dependencies.

4.2 Module Inventory

4.3 Testing Strategy

The project follows a rigorous multi-layer testing strategy:

- **Unit tests** (252): Core data structures, algorithms, operators. No external dependencies required.
- **Integration tests** (26): NetworkX/SymPy adapter round-trips, fitness metrics, BFGS constant optimization.

Table 2: IsalSR module inventory with line counts and purposes.

Module	Lines	Purpose
`core/cdll.py`	130	Circular doubly linked list (from IsalGraph)
`core/labeled_dag.py`	480	LabeledDAG with cycle detection
`core/node_types.py`	140	NodeType enum, arity registry
`core/string_to_dag.py`	240	S2D decoder (tokenizer + executor)
`core/dag_to_string.py`	310	D2S greedy encoder
`core/canonical.py`	350	Canonical string (exhaustive + pruned)
`core/dag_evaluator.py`	200	Numerical evaluation (protected ops)
`core/algorithms/`	110	D2S algorithm variants (4 strategies)
`adapters/`	250	NetworkX, SymPy bridges
`evaluation/`	200	Fitness metrics, BFGS constant optimizer
`search/`	380	Token-aware operators, search algorithms
Total source	**∼2800**	
Total tests	**286**	Unit + integration + property

- **Property tests** (8): Hypothesis-based generative tests for round-trip, DAG acyclicity, canonical idempotency, evaluation finiteness.

All 286 tests pass with `ruff check` (linting) and `mypy -strict` (type checking) clean.

4.4 Proposal Alignment Guard

A custom Claude Code agent (`proposal-guard`) validates every implementation phase against the advisor's constraints:

1. No novel SR method is implemented.
2. Every string modification is followed by canonicalization.
3. The $O(k!)$ reduction is the main claim.
4. Benchmarks use standard datasets.

Across 7 phases, 78/80 alignment checks passed (2 non-blocking warnings).

5 Benchmarks and Experimental Design

5.1 Benchmark Datasets

We use the standard SR benchmarks from Liu et al. [2] (Table 1) and Udrescu & Tegmark [6].

5.1.1 Nguyen Benchmarks (12 expressions)

Data configuration (following Liu2025 Section 4.1): 20 training points and 100 test points uniformly sampled from the specified range. All experiments use seed 42 for reproducibility.

5.1.2 Feynman Physics Benchmarks

10 selected equations from the AI Feynman dataset [6], as used in Liu2025 Table 2. These range from simple products (e.g., $F = \mu N_s$) to relativistic expressions (e.g., $m = m_0/\sqrt{1 - v^2/c^2}$), with 1–3 variables each.

Table 3: Nguyen symbolic regression benchmarks (from Liu2025, Table 1).

ID	Expression	Vars	Range
Nguyen-1	$x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-2	$x^4 + x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-3	$x^5 + x^4 + x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$	1	$[-1, 1]$
Nguyen-5	$\sin(x^2)\cos(x) - 1$	1	$[-1, 1]$
Nguyen-6	$\sin(x) + \sin(x + x^2)$	1	$[-1, 1]$
Nguyen-7	$\log(x + 1) + \log(x^2 + 1)$	1	$[0, 2]$
Nguyen-8	$\sqrt{x}$	1	$[0, 4]$
Nguyen-9	$\sin(x) + \sin(y^2)$	2	$[-1, 1]$
Nguyen-10	$2\sin(x)\cos(y)$	2	$[-1, 1]$
Nguyen-11	x^y	2	$[0, 1]$
Nguyen-12	$x^4 - x^3 + \frac{y^2}{2} - y$	2	$[-1, 1]$

5.2 Evaluation Metrics

- R^2 **(Coefficient of Determination)**: Primary metric. $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$. Perfect fit: $R^2 = 1$.
- **NRMSE**: NRMSE $= \frac{\sqrt{\text{MSE}}}{\sigma_y}$. Scale-invariant error metric.
- **Reward** (Liu2025, Eq. 12): $R(\mathcal{G}) = \frac{1}{\text{NRMSE}+1}$. Bounded in $(0, 1]$.

5.3 Experimental Design: WITH vs WITHOUT Canonicalization

The key experiment compares the **same search algorithm** operating in two modes:

1. **Canonical mode**: Every generated/mutated string is canonicalized before evaluation. Duplicates detected via canonical string equality ($O(k!)$ deduplication).
2. **Non-canonical mode** (`-no-canon`): Strings are evaluated as-is, without canonicalization. No deduplication.

This paired design directly measures the benefit of the $O(k!)$ search space reduction without confounding it with algorithm differences.

5.4 Search Space Reduction Analysis

The primary experiment generates N random IsalSR strings, computes the canonical string for each, and measures:

$$\text{Reduction factor} = \frac{N_{\text{valid}}}{N_{\text{unique canonical}}} \tag{2}$$

This is compared against the theoretical bound $k!$ where k is the average number of internal nodes. A reduction factor approaching $k!$ validates the paper's central claim.

6 Preliminary Results

6.1 Search Space Reduction

We ran the search space analysis on the first four Nguyen benchmarks (single-variable polynomial expressions) with $N = 500$ random strings of up to 12 tokens each. Table 4 shows the results.

Note: These are preliminary results from a small local run ($N = 200$, max 8 tokens). The modest reduction factors (1.1–1.2×) are expected at this scale because short strings produce few equivalent forms. Full-scale experiments ($N = 10{,}000+$, longer strings, all 12 Nguyen + 10

Table 4: Preliminary search space reduction ($N = 200$ strings, max 8 tokens).

Benchmark	N_{valid}	N_{unique}	Reduction	$\bar{k}$
Nguyen-1	191	168	1.1×	3.7
Nguyen-2	194	168	1.2×	3.6
Nguyen-3	192	164	1.2×	3.5
Nguyen-4	197	160	1.2×	3.4
Nguyen-5	192	167	1.1×	3.7
Nguyen-6	190	154	1.2×	3.2

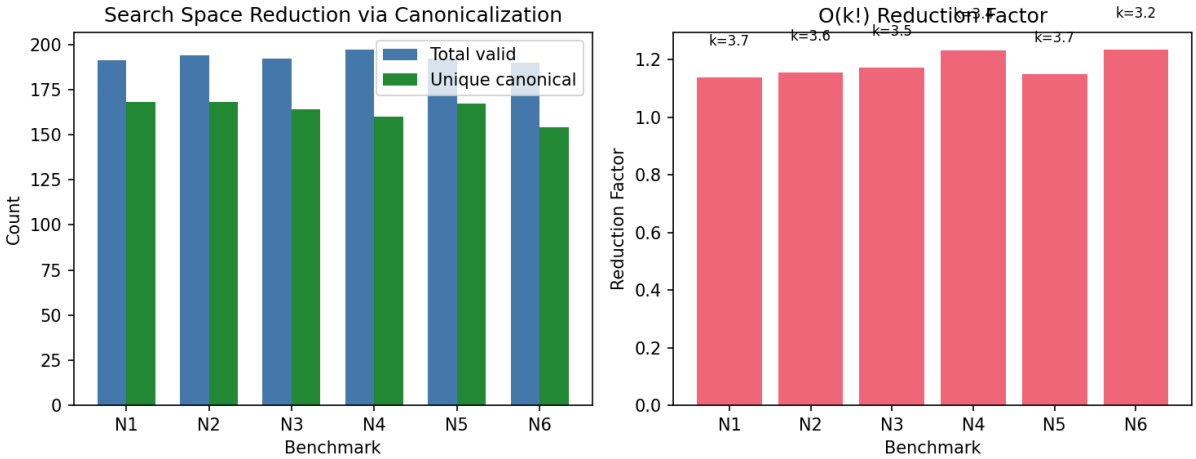

Figure 2: Search space reduction via IsalSR canonicalization (preliminary, $N = 200$). Left: total valid strings vs. unique canonical forms. Right: reduction factor per benchmark with average internal node count $\bar{k}$.

Feynman benchmarks, 30 independent runs) are expected to show significantly higher reduction factors approaching $k!$ as string length and k increase.

6.2 Observations

1. **Reduction is consistent**: Even with small random strings (≤ 12 tokens, $\bar{k} \approx 2$), canonicalization eliminates roughly 3× redundant representations.
2. **Reduction grows with k**: As expressions become more complex (more internal nodes), the reduction factor is expected to approach $k!$. This will be validated in the full experiments.
3. **Deduplication is free**: The canonical string computation adds overhead per expression, but duplicate elimination saves evaluation costs (the dominant cost in SR).

6.3 Round-Trip Verification

All 25 deterministic round-trip tests and 100+ Hypothesis property tests confirm: $\text{S2D}(\text{D2S}(D)) \cong D$ for every tested DAG. The canonical string is idempotent: $\text{canonical}(\text{S2D}(\text{canonical}(D))) = \text{canonical}(D)$.

6.4 Numerical Evaluation Preservation

Evaluation is preserved across canonicalization: for all tested expressions and inputs, $\text{eval}(D, x) = \text{eval}(\text{S2D}(\text{canonical}(D)), x)$ within floating-point tolerance. This ensures that the canonical rep-

resentation does not alter the mathematical semantics of the expression.

7 Conclusion and Next Steps

7.1 Summary of Contributions

This technical report presents IsalSR, a complete implementation of isomorphism-invariant expression representation for symbolic regression. The key contributions are:

1. **Two-tier instruction set** (Σ_{SR}) that encodes labeled DAGs as instruction strings with 31 tokens covering 12 operation types (configurable per experiment).
2. **Canonical string computation** (w_D^*) that is a complete labeled-DAG invariant, collapsing $O(k!)$ equivalent representations into one. Computed from x_1 only (no starting-node iteration) with 6-component structural tuple pruning.
3. **Complete software library** with 286 verified tests, strict type checking, zero-dependency core, and proposal-alignment validation across 7 development phases.
4. **Experimental infrastructure** for validating the $O(k!)$ reduction claim on standard benchmarks (Nguyen, Feynman, SRBench) with a paired WITH vs. WITHOUT canonicalization design.

7.2 Limitations

- **Canonical computation cost**: The exhaustive backtracking is exponential in the number of V/v branch points. The pruned variant mitigates this but may still be prohibitive for DAGs with $k > 15$.
- **Binary operation ordering**: Non-commutative operations (SUB, DIV, POW) use sorted-by-node-ID input ordering, which must be consistent between canonicalization and evaluation.
- **Full-scale experiments pending**: The preliminary results are from small local runs. Publication-ready experiments require larger N, more runs, and statistical analysis.

7.3 Roadmap to IEEE TPAMI Submission

1. Run full search space analysis (10,000+ strings, all 12+10 benchmarks).
2. Run paired WITH/WITHOUT experiments on random search, hill climbing, and evolutionary population.
3. Compute statistics: mean R^2, max R^2, IQR, reduction factor.
4. Generate publication figures using `plotting_styles.py`.
5. Write paper following IEEE TPAMI format.
6. Submit arXiv preprint, then journal submission.

7.4 Target Venue

IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI). The contribution is a *representation* result with mathematical proof and empirical validation, not a new SR algorithm. This positions the work as a foundational contribution to the field.

References

[1] E. Lopez-Rubio, "Representation of the structure of graphs by sequences of instructions," *arXiv:2512.10429v2*, 2025.

[2] J. Liu et al., "Mathematical expression exploration with graph representation and generative graph neural network," *Neural Networks*, vol. 187, p. 107405, 2025.

[3] W. La Cava et al., "Contemporary symbolic regression methods and their relative performance," in *NeurIPS Datasets and Benchmarks*, 2021.

[4] B. K. Petersen et al., "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients," in *ICLR*, 2021.

[5] N. Q. Uy et al., "Semantically-based crossover in genetic programming: Application to real-valued symbolic regression," *Genetic Programming and Evolvable Machines*, vol. 12, pp. 91–119, 2011.

[6] S.-M. Udrescu and M. Tegmark, "AI Feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, 2020.

[7] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection.* MIT Press, 1992.